

Sistema de Gestão de Contas Financeiras

1. Visão Geral do Sistema

1.1. Objetivo

O Sistema de Gestão de Contas é uma aplicação web Full-Stack projetada para simular um ambiente bancário simplificado. O seu principal objetivo é fornecer uma plataforma para a gestão de clientes (pessoas) e das suas respectivas contas financeiras, permitindo a execução de operações bancárias essenciais e a extração de relatórios analíticos.

1.2. Principais Funcionalidades

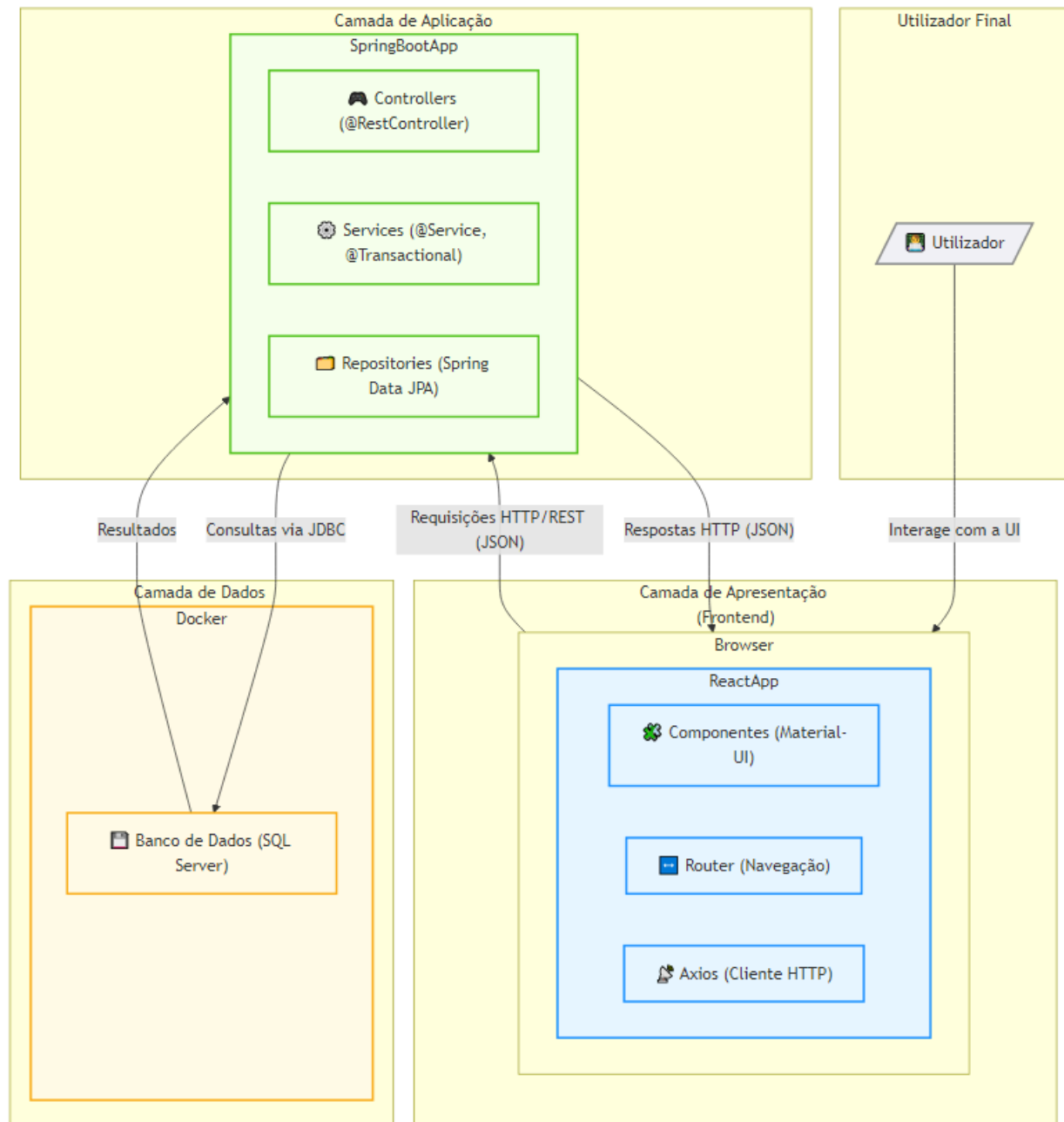
- **Gestão de Clientes (Pessoas):** CRUD completo (Create, Read, Update, Delete) para o registo de clientes, com validações de negócio como a unicidade do CPF.
- **Gestão de Contas:** Criação de contas (corrente ou poupança) associadas a um cliente e a capacidade de alterar o status de uma conta (ativa/bloqueada).
- **Operações Financeiras:** Execução de transações atômicas e seguras, incluindo Depósito, Saque e Transferência entre contas.
- **Consultas e Extratos:** Acesso ao saldo em tempo real e ao extrato de transações de cada conta.
- **Relatórios Analíticos:** Geração de relatórios de inteligência de negócio, como o saldo médio de todos os clientes e o ranking dos "Top 10" clientes por volume de movimentação.

2. Arquitetura

O sistema adota uma arquitetura de três camadas (Three-Tier Architecture), garantindo uma separação clara de responsabilidades.

2.1. Camadas

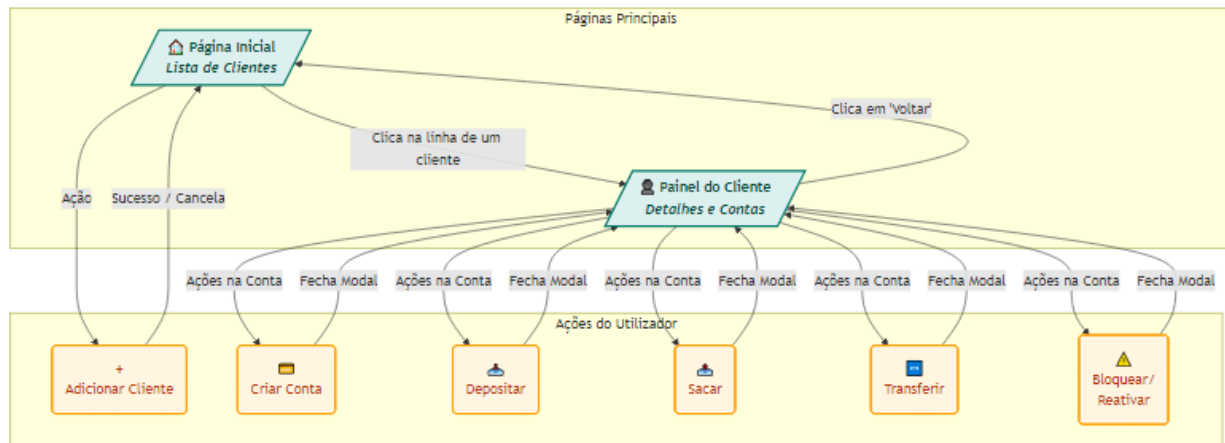
- **Frontend (Camada de Apresentação):**
 - Desenvolvido em **React** com **Vite**. É responsável por toda a interface do utilizador e pela experiência de interação;
 - Utiliza a biblioteca **Material-UI** para a componentização visual e o **Axios** para a comunicação com o backend.
- **Backend (Camada de Negócio):**
 - Construído com **Java** e **Spring Boot**;
 - Segue uma arquitetura em camadas (Controller, Service, Repository) para organizar a lógica de negócio, a gestão de dados e a exposição dos serviços.
- **Banco de Dados (Camada de Dados):**
 - Utiliza o **Microsoft SQL Server**, a correr num container Docker para facilitar a configuração do ambiente.



2.2. Comunicação

A comunicação entre o frontend e o backend é realizada através de uma **API RESTful**. O frontend envia pedidos HTTP para os endpoints expostos pelo backend, e os dados são trocados no formato **JSON**. A segurança de comunicação entre as diferentes origens (localhost:5173 e localhost:8080) é gerida via **CORS (Cross-Origin Resource Sharing)**, configurado no backend.

2.3. Fluxo de Funcionamento



3. Configuração e Execução

Para executar o projeto localmente, são necessários dois ambientes: um para o backend e um para o frontend.

3.1. Pré-requisitos

- Java JDK 17 ou superior
- Node.js e npm
- Docker Desktop

3.2. Passos para Execução

1. Configurar o application.properties (backend):

Garanta que as seguintes linhas estão no arquivo, com a senha correta:

```
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=db_gestao_contas;encrypt=false;trustServerCertificate=true;
spring.datasource.username=sa
spring.datasource.password=DesafioAPI123!
spring.jpa.hibernate.ddl-auto=validate
```

2. **Subir o SQL Server (Docker):**

```
docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=DesafioAPI123!" -p 1433:1433 --name sqlserver-desafio -d [mcr.microsoft.com/mssql/server:2022-latest](https://mcr.microsoft.com/mssql/server:2022-latest)
```

3. Migrar e Popular o Banco:

Use um cliente de SQL (DBeaver, Azure Data Studio) para se conectar ao container e executar o script V1__create_tables.sql.

4. Executar o Backend (Terminal 1):
Navegue até à pasta gestaocontas-api e execute:
`.\mvnw.cmd spring-boot:run`
5. Executar o Frontend (Terminal 2):
Navegue até à pasta gestaocontas-ui e execute:
`npm install` # Apenas na primeira vez
`npm run dev`
6. Rodar Testes Unitários (Backend):
No terminal do backend, execute:
`.\mvnw.cmd test`

O resultado BUILD SUCCESS indica que todos os testes passaram.

4. Endpoints da API

A API está documentada e pode ser testada interativamente através do Swagger UI.

- **URL do Swagger:** <http://localhost:8080/swagger-ui.html>

Abaixo estão alguns dos principais endpoints:

Pessoas (/pessoas)

- GET /: Lista todas as pessoas.
- GET /{id}: Busca uma pessoa por ID.
- POST /: Cria uma nova pessoa.
 - **Request Body:** { "nome": "string", "cpf": "string", "dataNascimento": "YYYY-MM-DD" }
 - **Success Response (201):** { "idPessoa": number, "nome": "string", ... }

Contas (/contas)

- POST /: Cria uma nova conta.
 - **Request Body:** { "idPessoa": number, "limiteSaqueDiario": number, "tipoConta": number }
- GET /pessoa/{idPessoa}: Lista todas as contas de uma pessoa.
- POST /{idConta}/deposito: Realiza um depósito.
 - **Request Body:** { "valor": number }
- PATCH /{idConta}/alterar-status: Alterna o status da conta (ativa/bloqueada).

5. Estrutura do Banco de Dados

O esquema é composto por três tabelas principais, com nomes e colunas a seguir o padrão `snake_case` para alinhar com as convenções do Spring Boot. Esta padronização foi adotada após a fase inicial de desenvolvimento, onde foi detectado um conflito com o comportamento padrão do Hibernate no ecossistema Spring Boot. Inicialmente, o esquema do banco de dados foi criado utilizando a convenção `camelCase` (ex: `idConta`). No entanto, durante a inicialização da aplicação, o Hibernate lançava uma `SchemaManagementException` com a mensagem `Schema-validation: missing column [id_conta]`. A investigação revelou que a estratégia de nomeação física padrão do Spring (`SpringPhysicalNamingStrategy`) converte automaticamente os nomes de atributos `camelCase` das entidades JPA para o formato `snake_case` antes de gerar as consultas SQL de validação.

tb_pessoa			
BIGINT	id_pessoa	PK	Chave Primária
VARCHAR	nome		VARCHAR(255)
VARCHAR	cpf		VARCHAR(11), UNIQUE
DATE	data_nascimento		

possui

tb_conta			
BIGINT	id_conta	PK	Chave Primária
BIGINT	id_pessoa	FK	Chave Estrangeira
DECIMAL	saldo		DECIMAL(18,2)
DECIMAL	limite_saque_diario		DECIMAL(18,2)
BIT	flag_ativo		1=Ativa, 0=Bloqueada
INT	tipo_conta		1=Corrente, 2=Poupança
DATETIMEOFFSET	data_criacao		

registra

tb_transacao			
BIGINT	id_transacao	PK	Chave Primária
BIGINT	id_conta	FK	Chave Estrangeira
DECIMAL	valor		DECIMAL(18,2), Positivo=Entrada, Negativo=Saída
DATETIMEOFFSET	data_transacao		

- `tb_pessoa`: Armazena os dados dos clientes;
- `tb_conta`: Armazena os dados das contas bancárias. Relaciona-se com `tb_pessoa` (uma pessoa pode ter várias contas);
- `tb_transacao`: Armazena o registro de todas as movimentações. Relaciona-se com `tb_conta` (uma conta pode ter várias transações).

6. Detalhes do Backend

- **Controllers (@RestController)**

- **Responsabilidade:** São a porta de entrada da API. Mapeiam os endpoints HTTP (ex: POST /pessoas) para os métodos de serviço correspondentes.
- **Fluxo:** Recebem as requisições, desserializam o corpo (JSON) para DTOs, validam os dados de entrada com @Valid e delegam a execução para a camada de Service. No final, formatam a resposta (sucesso ou erro) e a enviam de volta com o código de status HTTP apropriado.

- **Services (@Service)**

- **Responsabilidade:** O coração da aplicação, onde reside toda a lógica de negócio. Orquestram as operações e garantem a integridade dos dados.
- **Transacionalidade (@Transactional):** Métodos críticos, como transferir() e sacar(), são anotados com @Transactional. Isto garante a **atomicidade**: se qualquer passo da operação falhar, todas as alterações no banco de dados são revertidas (rollback), prevenindo inconsistências.

- **Repositories (Spring Data JPA)**

- **Responsabilidade:** Camada de abstração para o acesso a dados. São interfaces que estendem JpaRepository.
- **Funcionamento:** O Spring Data JPA cria a implementação em tempo de execução, traduzindo nomes de métodos (ex: findByCpf(String cpf)) ou anotações @Query em consultas SQL otimizadas, abstraindo a complexidade do JDBC.

- **DTOs (Data Transfer Objects)**

- **Responsabilidade:** Atuam como um "contrato" entre o frontend e o backend. São usados para receber dados (RequestDTO) e para enviar dados (ResponseDTO).
- **Benefícios:** Desacoplam a API do modelo de dados interno (Entidades JPA), previnem o vazamento de informações da base de dados e permitem a aplicação de validações específicas para a camada de API.

- **Mappers (MapStruct)**

- **Responsabilidade:** Automatizar a conversão entre DTOs e Entidades (Model).
- **Funcionamento:** A partir de uma simples interface, o MapStruct gera, em tempo de compilação, o código de conversão, eliminando código repetitivo, garantindo a segurança de tipos e oferecendo alta performance, pois não usa reflection.

- **Model (Entidades JPA)**

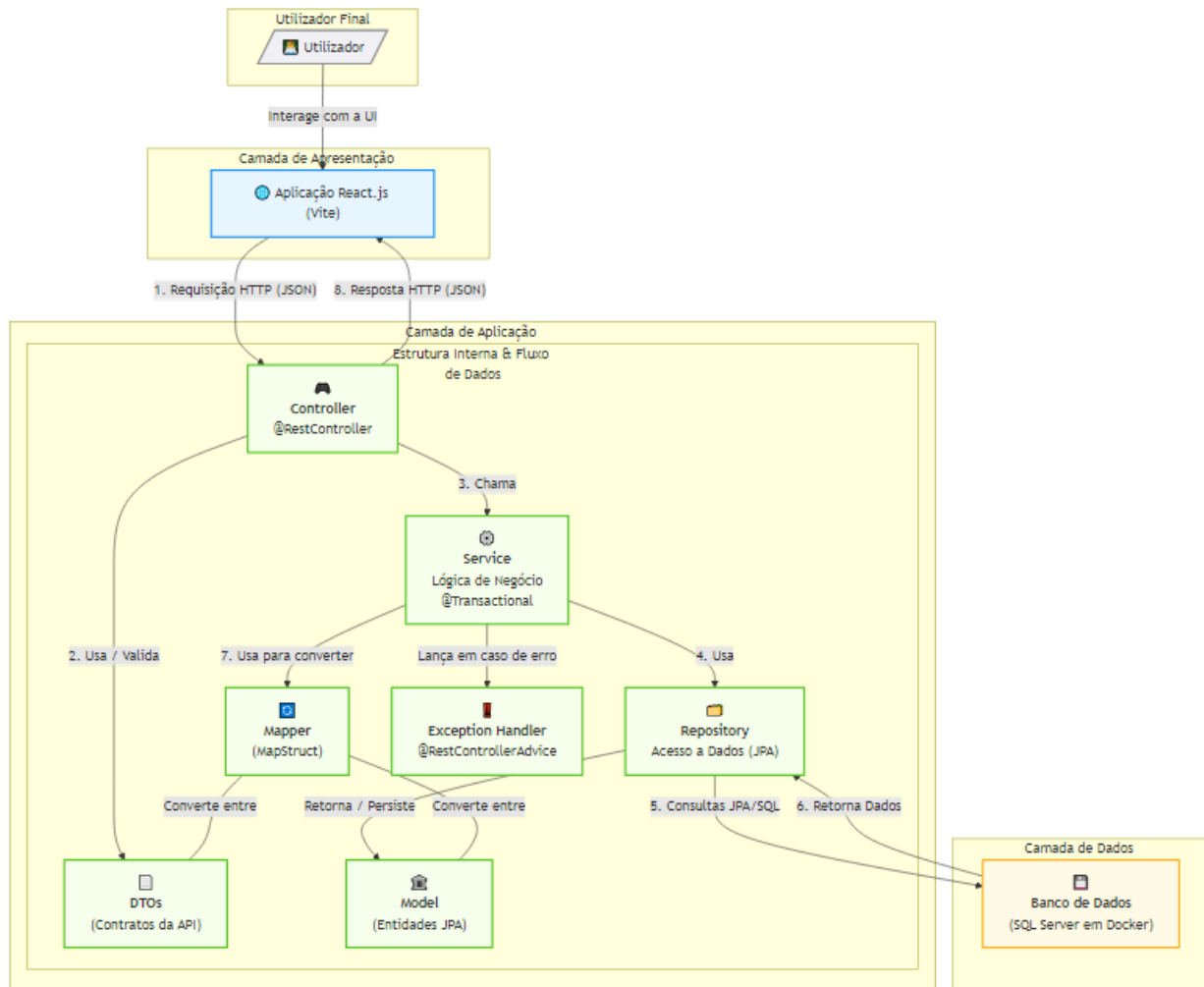
- **Responsabilidade:** Mapear a estrutura das tabelas do banco de dados para objetos Java.
- **Tecnologia:** Usam anotações da Jakarta Persistence API (`@Entity`, `@Table`, `@Column`, `@Id`, etc.) para que o Hibernate (a nossa implementação do JPA) saiba como persistir e buscar os dados.

- **Exception Handling (@RestControllerAdvice)**

- **Responsabilidade:** Centralizar o tratamento de exceções em toda a aplicação.
- **Fluxo:** Quando um Service lança uma exceção customizada (ex: `ResourceNotFoundException`), o `GlobalExceptionHandler` a captura e a traduz numa resposta JSON padronizada e amigável para o frontend, com o código de status HTTP correto (ex: 404 Not Found).

ContaServiceImplTest (JUnit + Mockito)

- **Responsabilidade:** Validar o comportamento da camada de serviço `ContaServiceImpl`, garantindo que regras de negócio sejam aplicadas corretamente durante operações de saque.
- **Funcionamento:**
 - Usa **JUnit 5** para estruturar os testes unitários;
 - Usa **Mockito** para simular dependências externas (`ContaRepository` e `TransacaoRepository`), isolando a lógica de negócio;
 - O método `@BeforeEach setUp()` cria uma conta de teste com saldo e limite configurados;
 - Os testes verificam cenários críticos:
 - Saldo insuficiente: deve lançar `IllegalOperationException` e não salvar alterações no repositório.
 - Conta bloqueada (`flagAtivo = false`): também deve lançar `IllegalOperationException`.



7. Detalhes do Frontend

- **Estrutura de Pastas**
 - **pages/:** Contém os componentes de "ecrã inteiro", que representam as principais páginas da aplicação (ListaPessoas, DetalhesCliente).
 - **components/:** Contém componentes reutilizáveis, como os formulários (CriarPessoaForm, TransacaoForm) e o Layout principal.
 - **api/:** Isola toda a lógica de comunicação com o backend num único ficheiro (apiService.js), tornando os componentes mais limpos e fáceis de testar.
- **Navegação (React Router DOM)**
 - A navegação entre as páginas é gerida pelo react-router-dom;
 - O componente App.jsx define as rotas da aplicação (ex: a rota /clientes/:id renderiza o componente DetalhesCliente);
 - A navegação é acionada tanto por componentes de link (<Link>) como de forma programática, através do hook useNavigate, como na função handleClick.
- **Gestão de Estado (React Hooks)**
 - **useState:** É o principal hook usado para gerir o estado local de cada componente, como os dados de um formulário, a lista de pessoas, ou o estado de visibilidade de um modal.
 - **useEffect:** Utilizado para executar "efeitos colaterais", principalmente para buscar dados da API (fetchData) assim que um componente é montado no ecrã.
 - **useCallback e useMemo:** Hooks de otimização usados para "memorizar" funções e valores calculados, respectivamente. Isto previne re-renderizações desnecessárias e melhora a performance, especialmente em componentes complexos como o DetalhesCliente.
- **UI e Estilização (Material-UI e Theming)**
 - Toda a UI é construída com base nos componentes pré-fabricados do **Material-UI** (Button, Card, Table, Dialog, etc.);
 - O ficheiro theme.js centraliza a identidade visual da aplicação.
- **Comunicação com a API (Axios)**
 - O ficheiro **apiService.js** cria uma instância do axios com a URL base do backend.
 - Ele exporta funções claras e nomeadas para cada endpoint (ex: getPessoas(), criarConta(data)), abstraindo os detalhes dos pedidos HTTP. Os componentes da UI apenas importam e chamam estas funções, sem precisarem de saber os URLs ou os métodos.